



D Data
S Structures
A Algorithms



Data Structures & Algorithms

Today's Topic: AVL Tree



Dr. Si Thu Aung



in/drsithuaung/

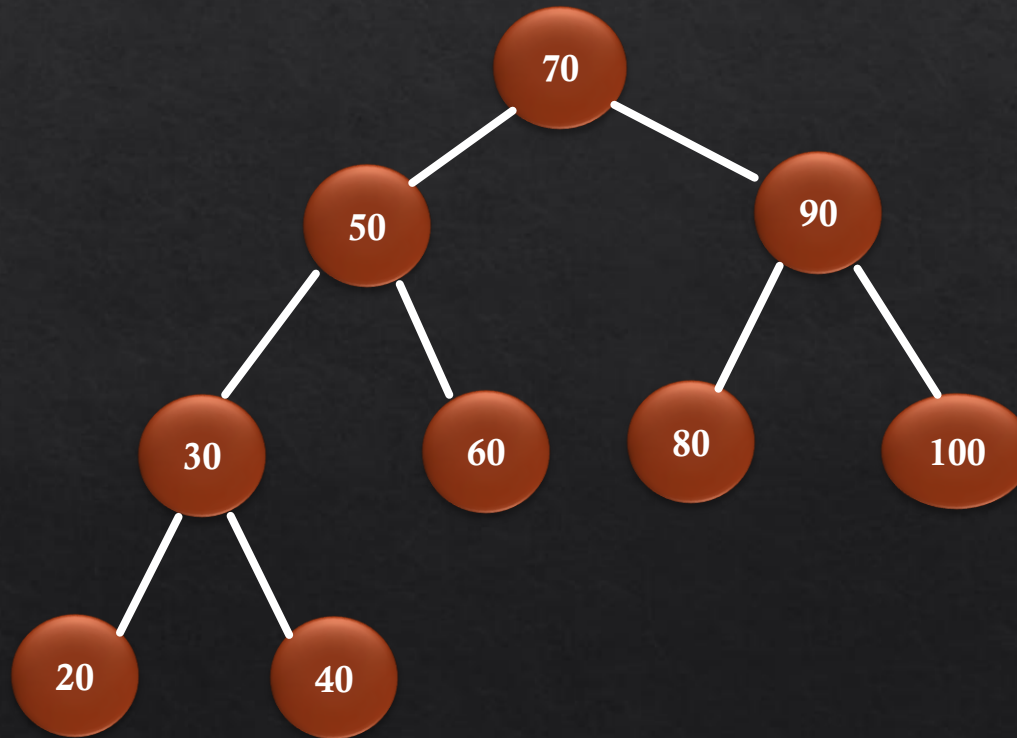


<https://sithuaung-ink.github.io/data-analyst-portfolio/>

4/7/2026

What is an AVL Tree?

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between the heights of left and right subtrees cannot be more than one for all nodes.



What is an AVL Tree?

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between the heights of left and right subtrees cannot be more than one for all nodes.

70

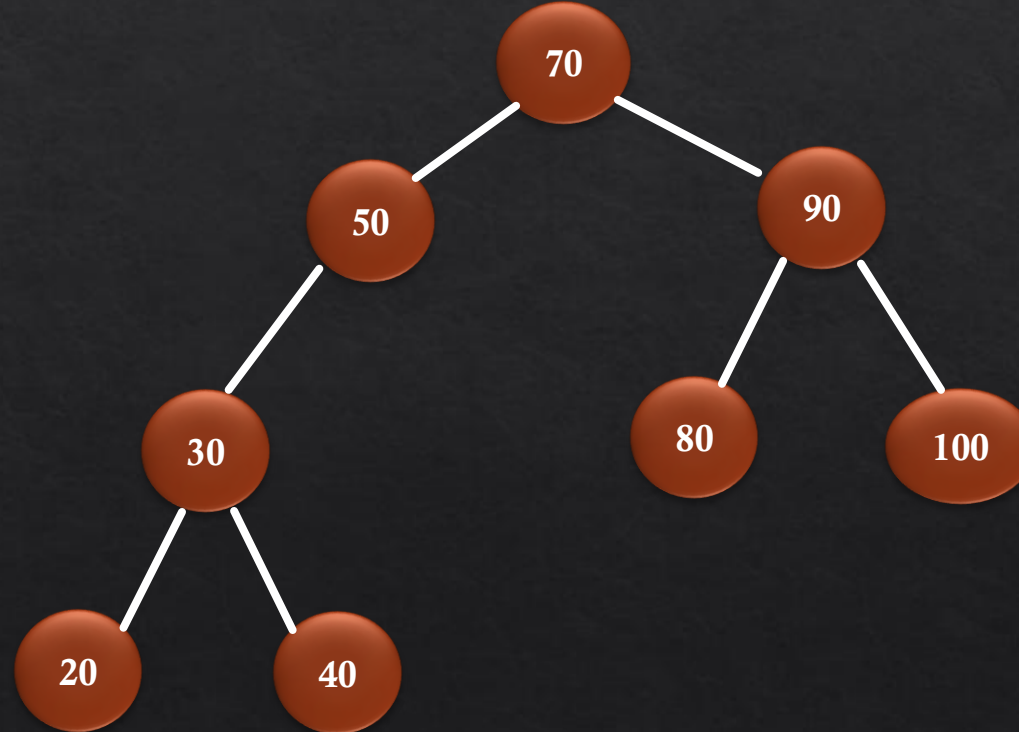
Height of left Subtree = 3
Height of right Subtree = 2
difference = 1

90

Height of left Subtree = 1
Height of right Subtree = 1
difference = 0

50

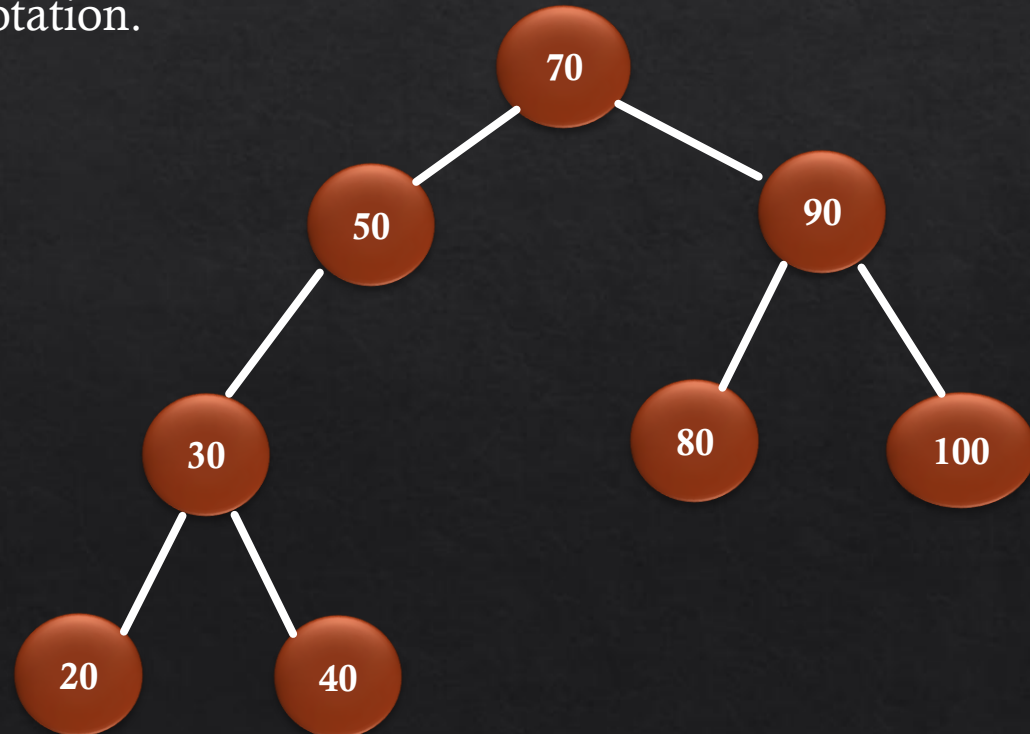
Height of left Subtree = 2
Height of right Subtree = 0
difference = 2



What is an AVL Tree?

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between the heights of left and right subtrees cannot be more than one for all nodes.

If at any time the heights of the left and right subtrees differ by more than 1, rebalancing is performed to restore the AVL property; this process is called a rotation.



What is an AVL Tree?

70

Height of left Subtree = 2

Height of right Subtree = 2

difference = 0

50

Height of left Subtree = 1

Height of right Subtree = 1

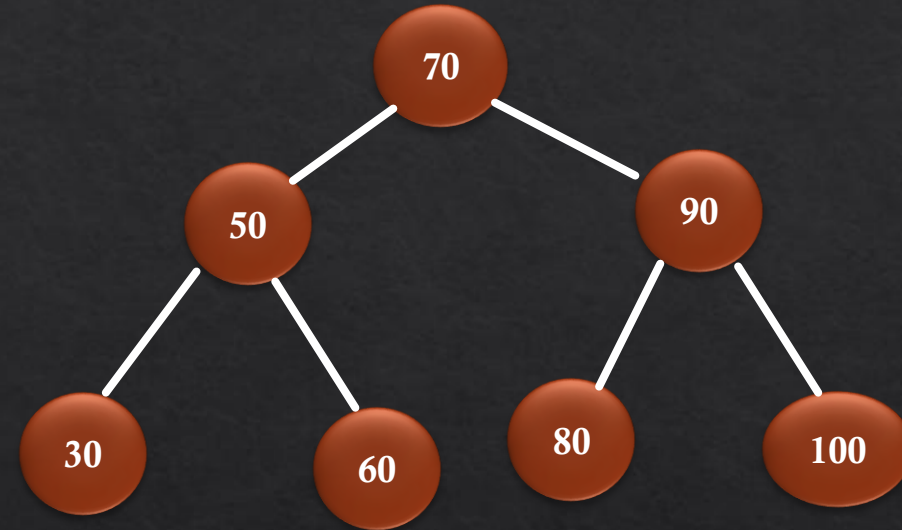
difference = 0

90

Height of left Subtree = 1

Height of right Subtree = 1

difference = 0



What is an AVL Tree?

70

Height of left Subtree = 3

Height of right Subtree = 2 difference = 1

50

Height of left Subtree = 2

Height of right Subtree = 1 difference = 1

90

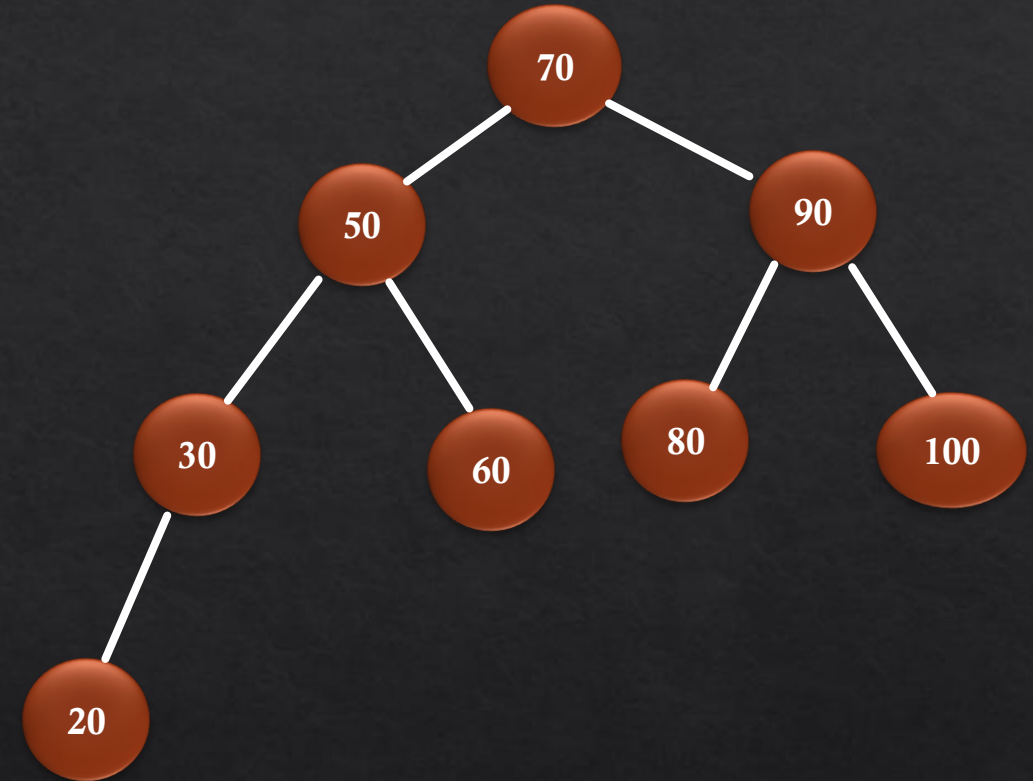
Height of left Subtree = 1

Height of right Subtree = 1 difference = 0

30

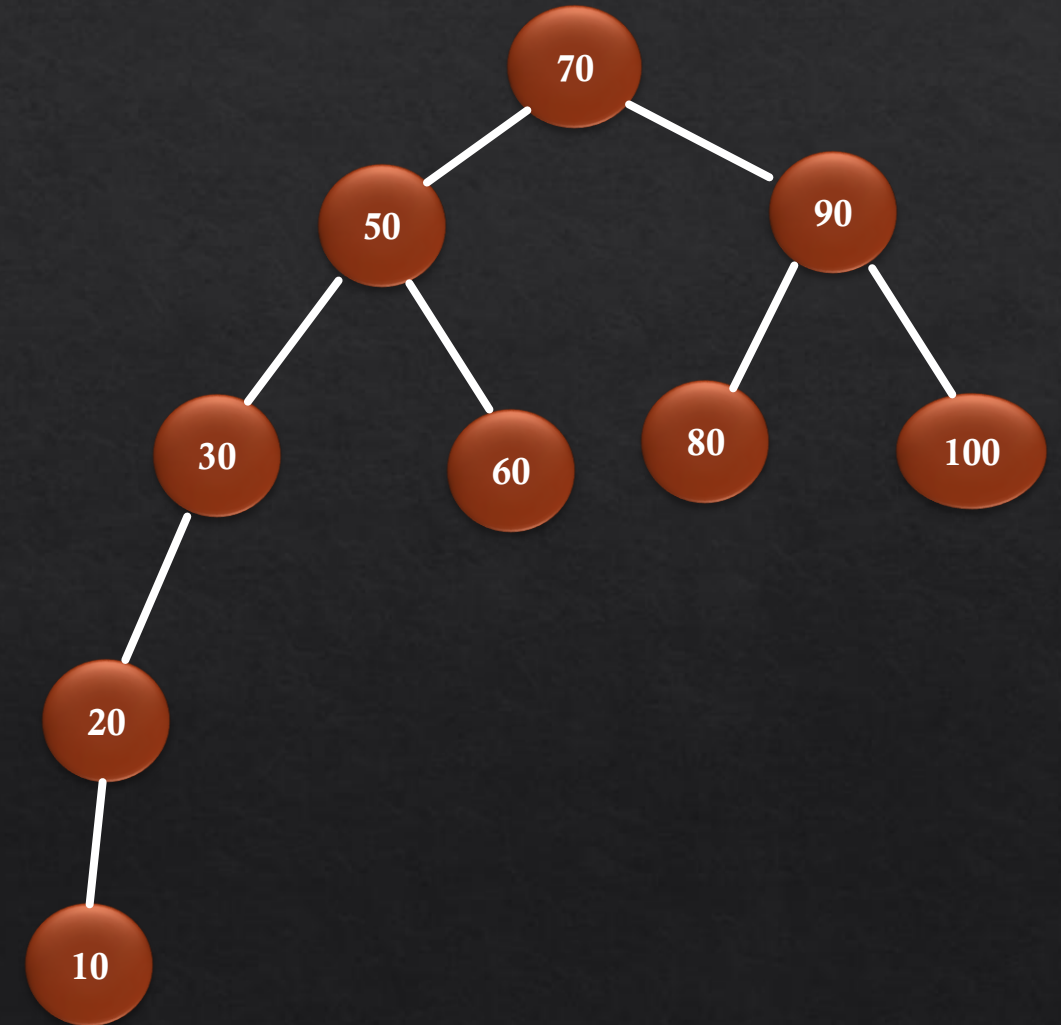
Height of left Subtree = 1

Height of right Subtree = 0 difference = 0



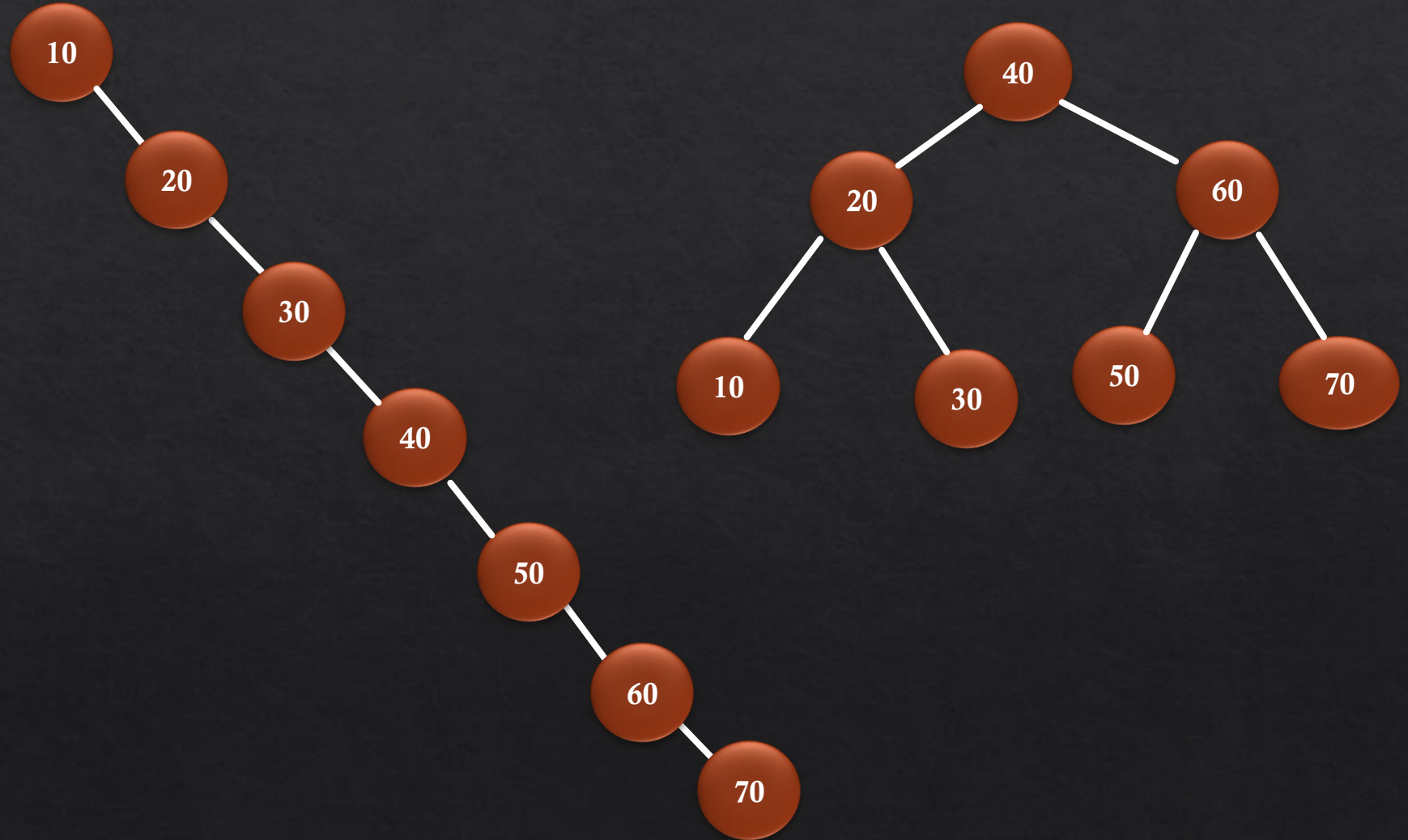
What is an AVL Tree?

70 Height of left Subtree = 4
Height of right Subtree = 2 difference = 2



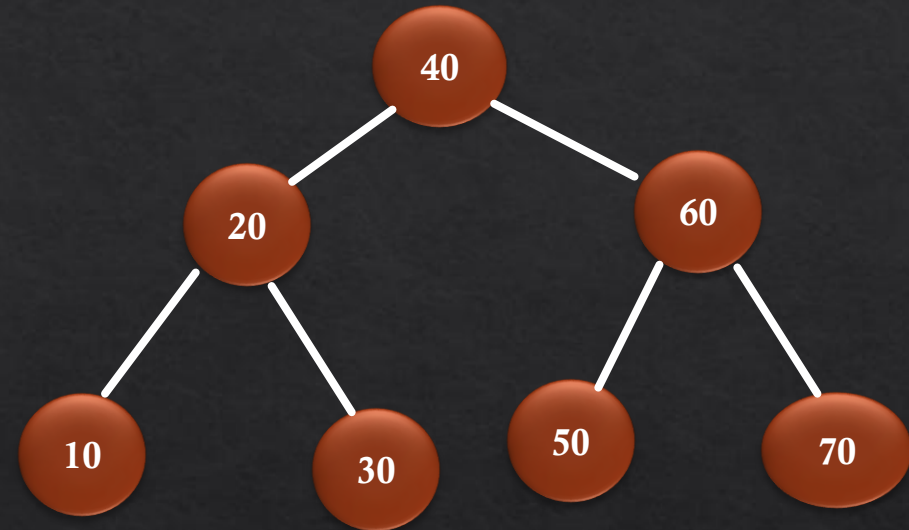
Why do we need an AVL Tree?

10, 20, 30, 40, 50, 60, 70



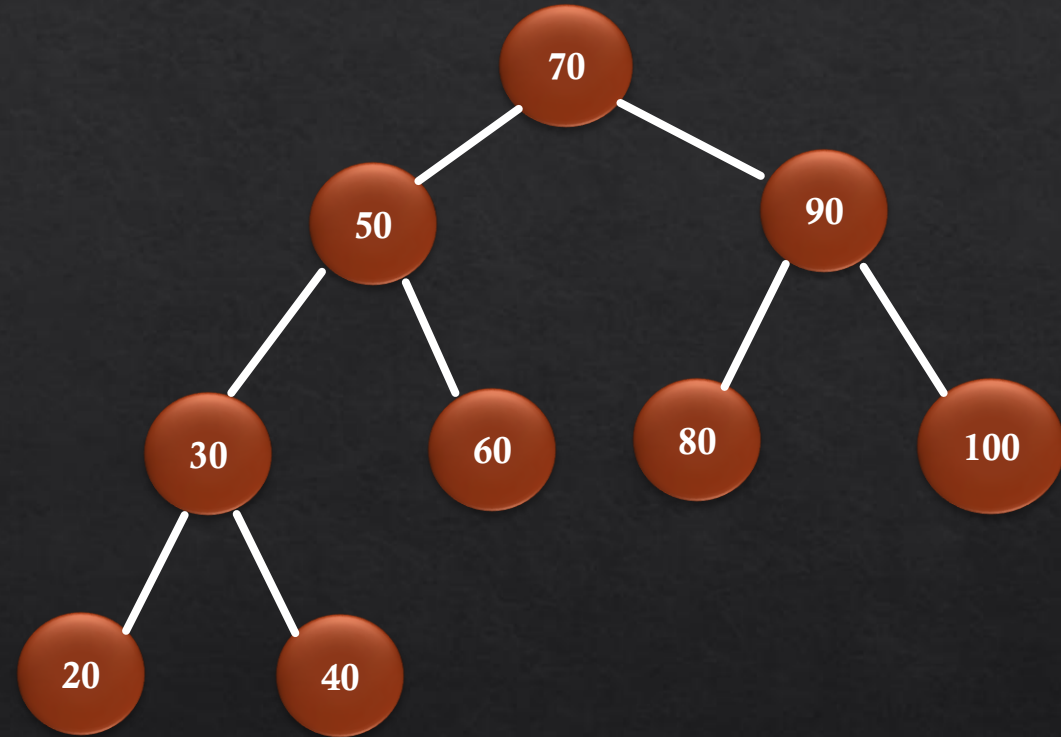
Operations of an AVL Tree

- Creation of AVL tree
- Search for a node in an AVL Tree
- Traverse all nodes in the AVL Tree
- Insertion of a node in an AVL Tree
- Deletion of a node in an AVL Tree
- Deletion of the tree



Creation of AVL Tree

```
newAVL = AVL()  
rootNode = None  
leftChild = None  
rightChild = None
```



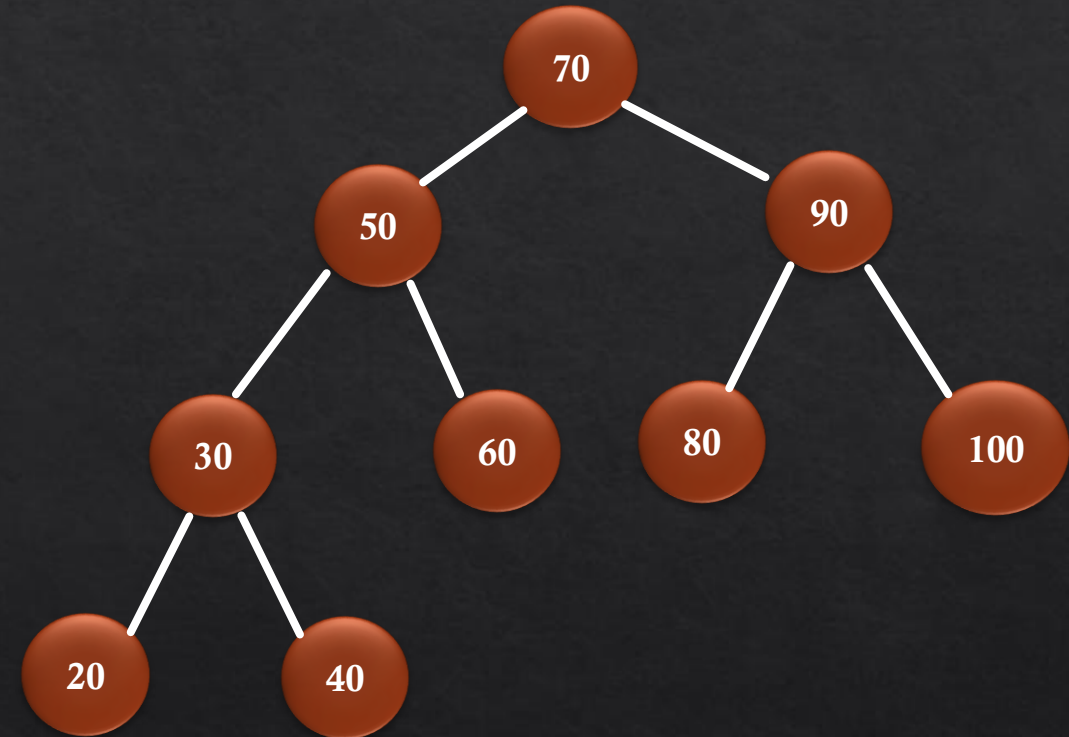
Traversal of an AVL Tree

Depth first search

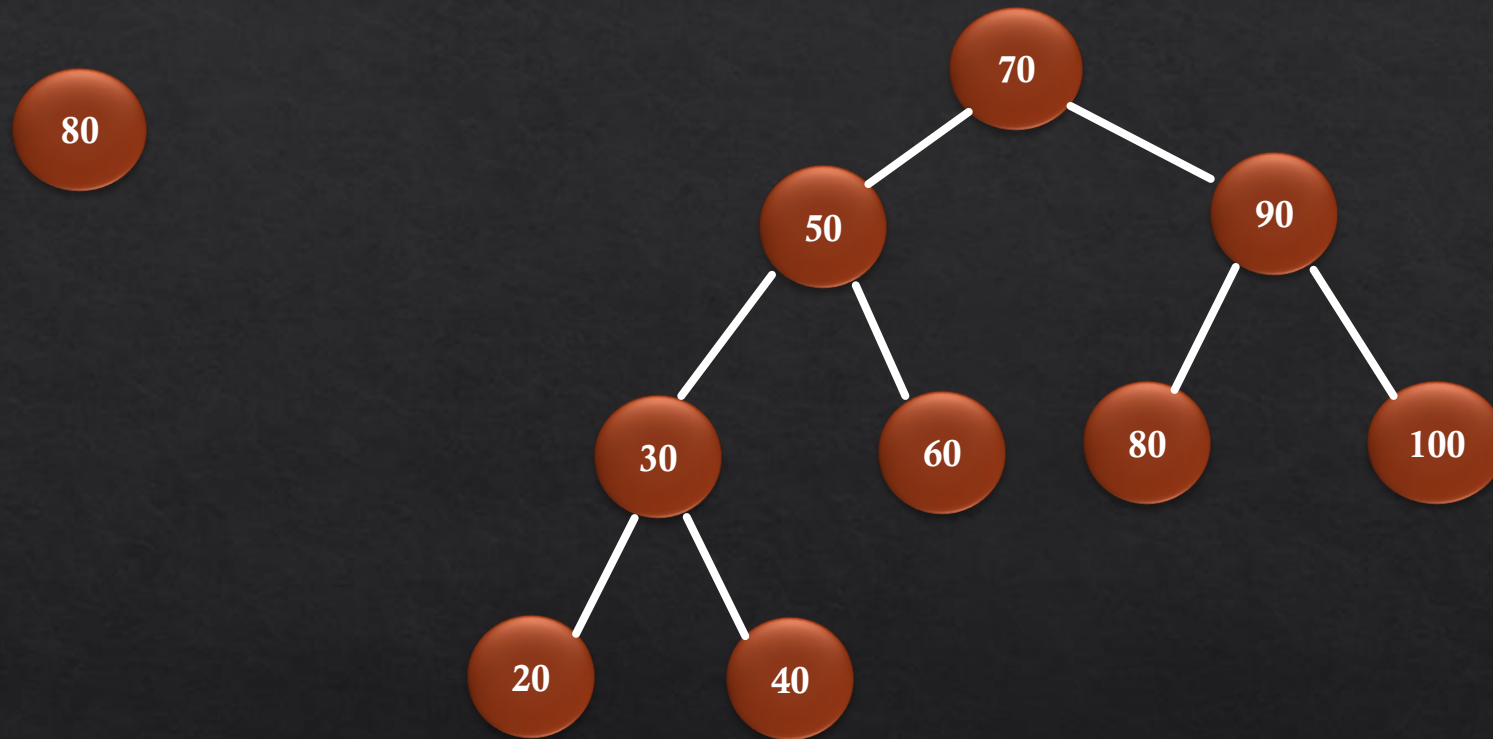
- Preorder traversal
- Inorder traversal
- Post Order traversal

Breadth first search

- Level order traversal



Search for a node in an AVL Tree



Insert a node in an AVL Tree

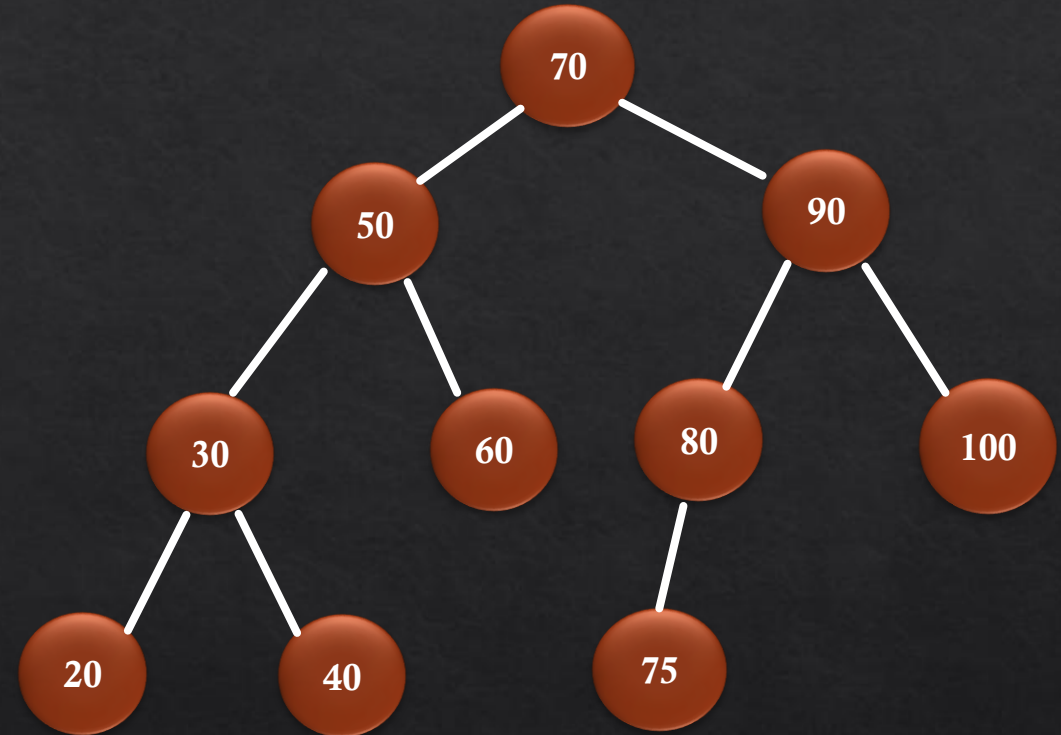
Case 1: Rotation is not required

Case 2: Rotation is required

Case 1: Rotation is not required

Case 2: Rotation is required

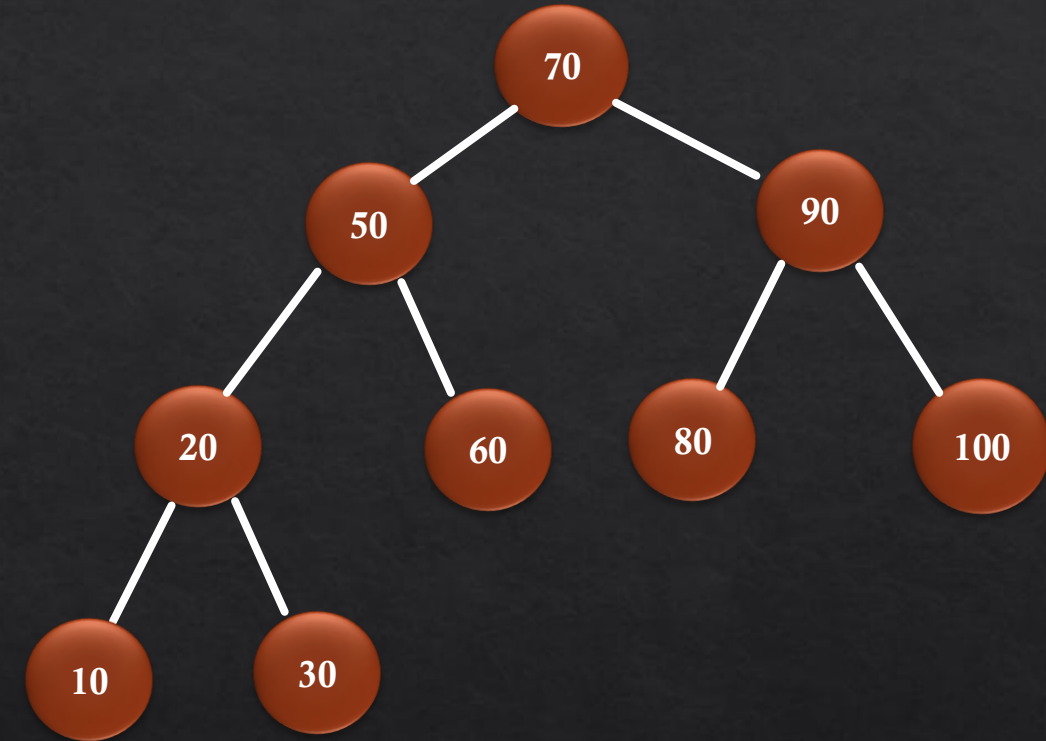
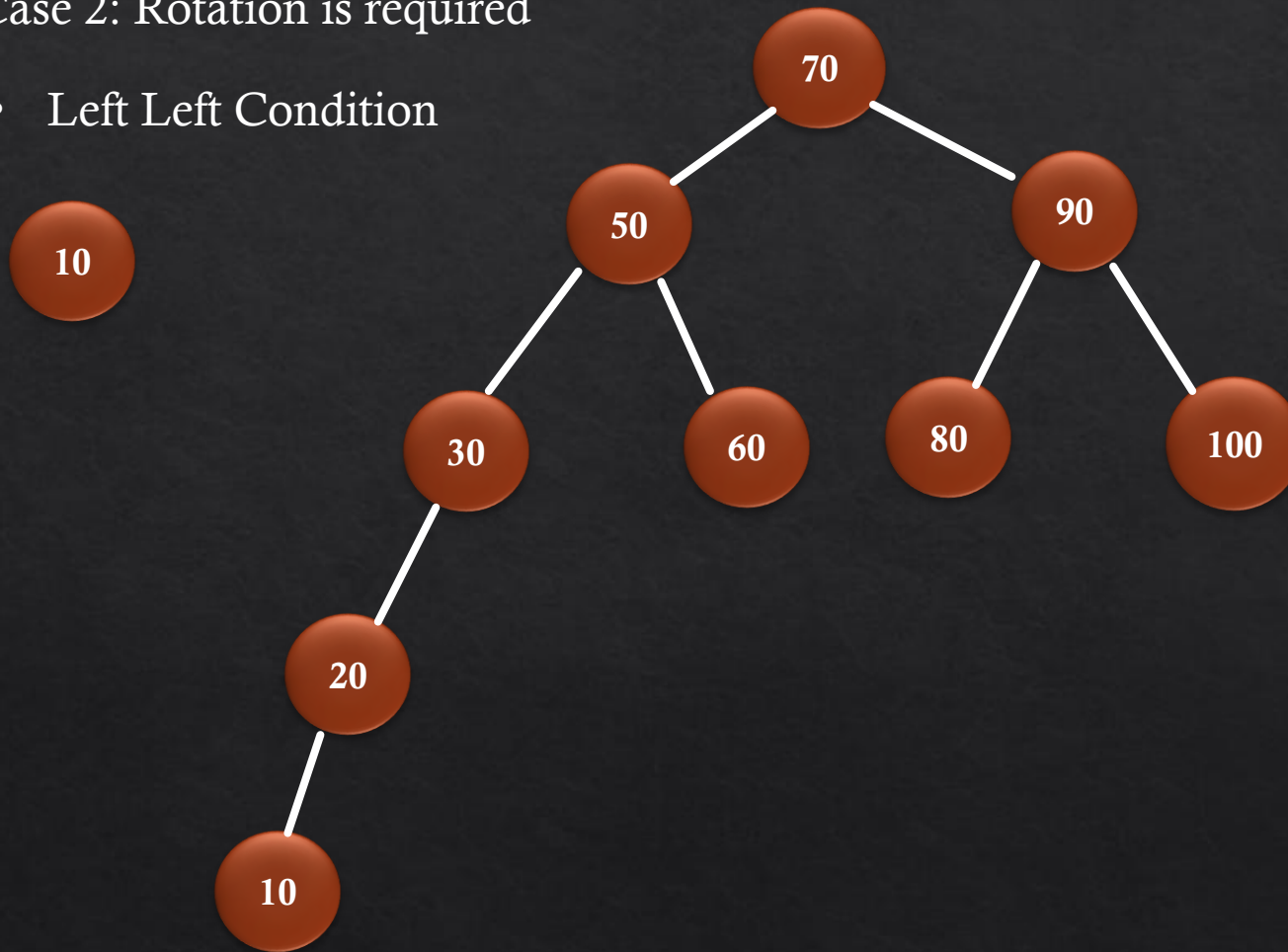
- LL - Left Left Condition
- LR - Left Right Condition
- RR - Right Right Condition
- RL - Right Left Condition



Insert a node in an AVL Tree

Case 2: Rotation is required

- Left Left Condition

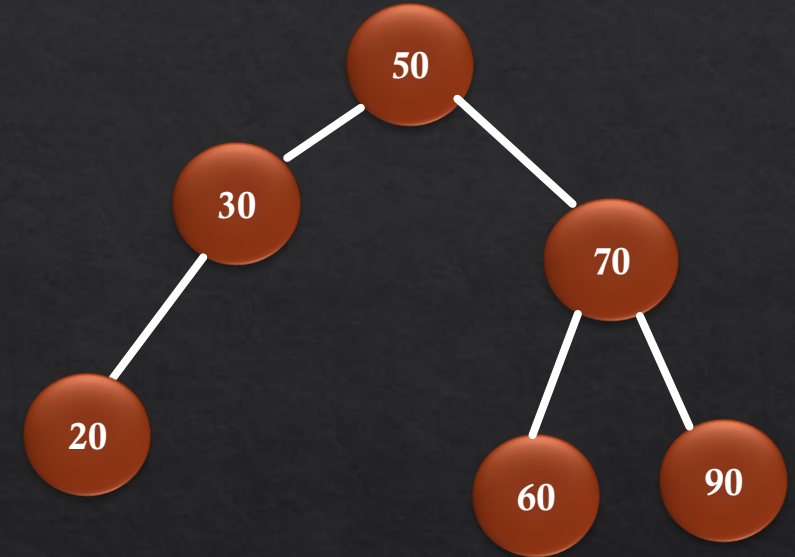
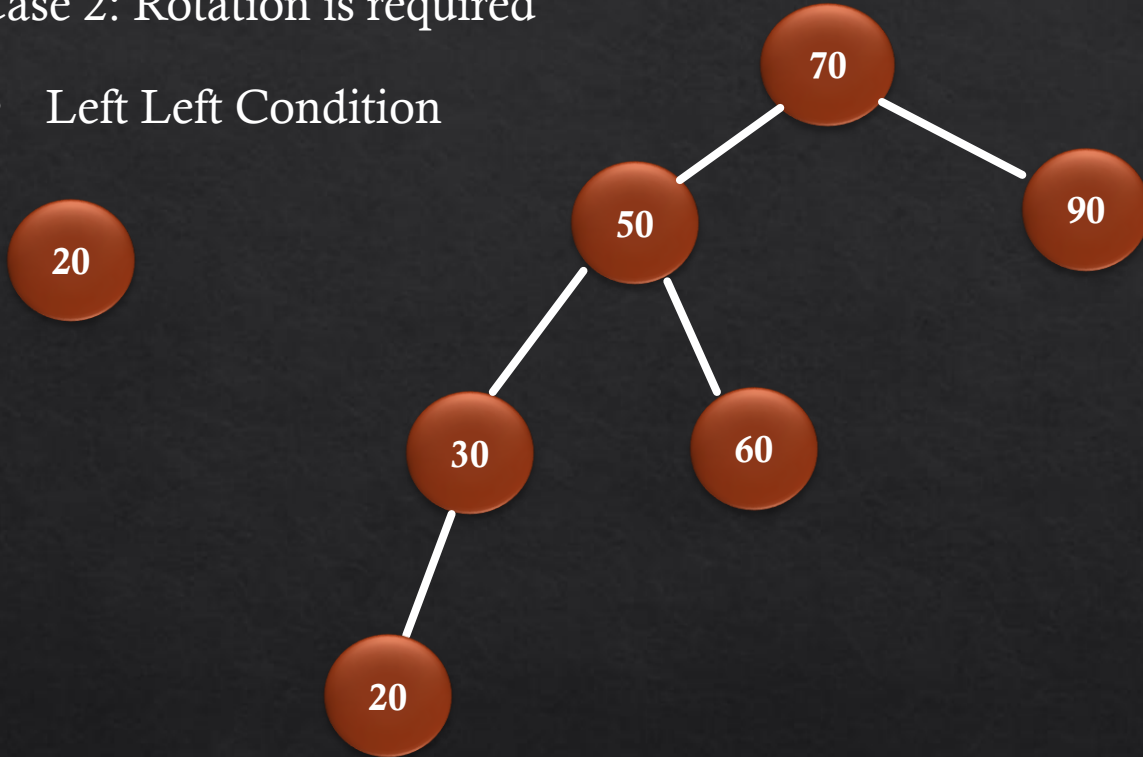


Right Rotation

Insert a node in an AVL Tree

Case 2: Rotation is required

- Left Left Condition

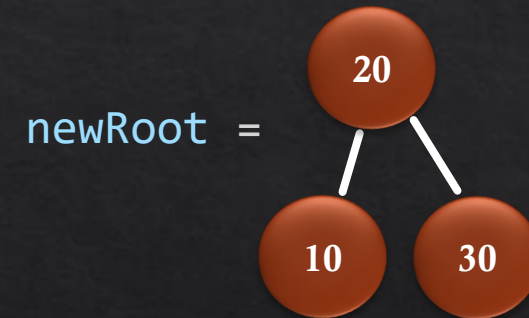
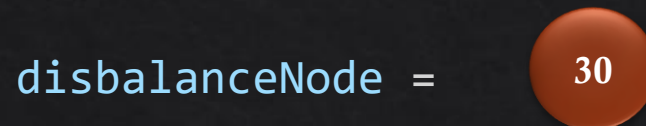
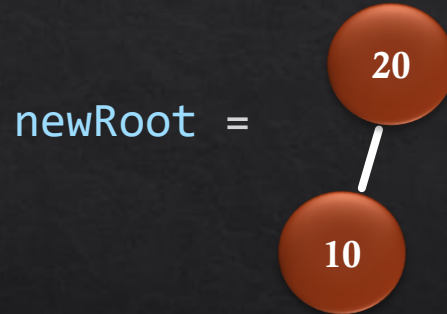
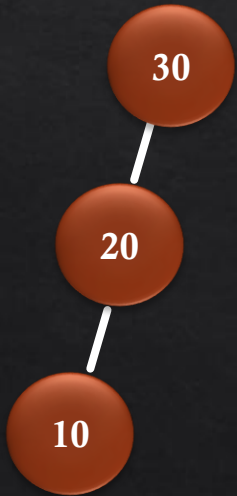


Right Rotation

Insert a node in an AVL Tree

Case 2: Rotation is required Left Left Condition

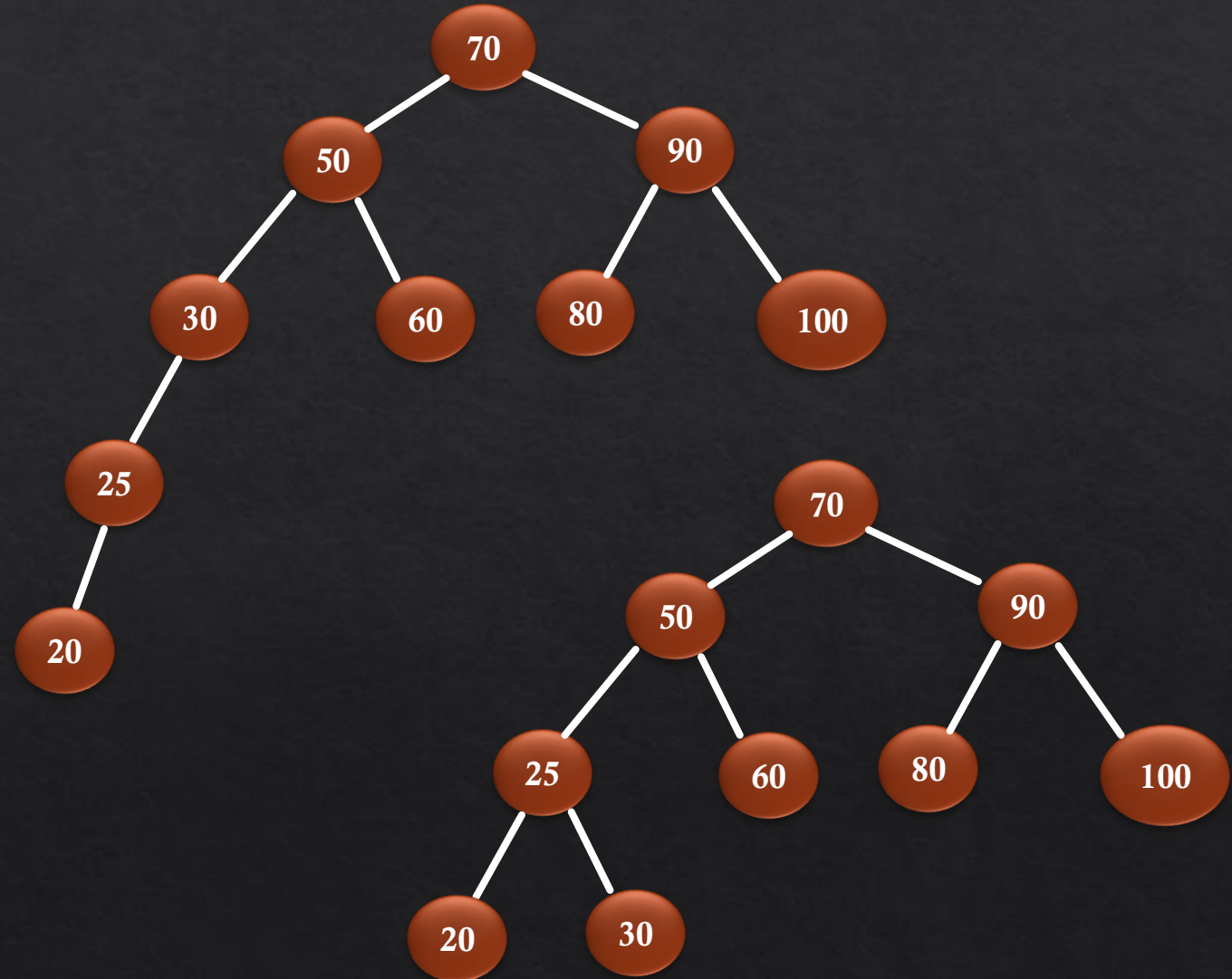
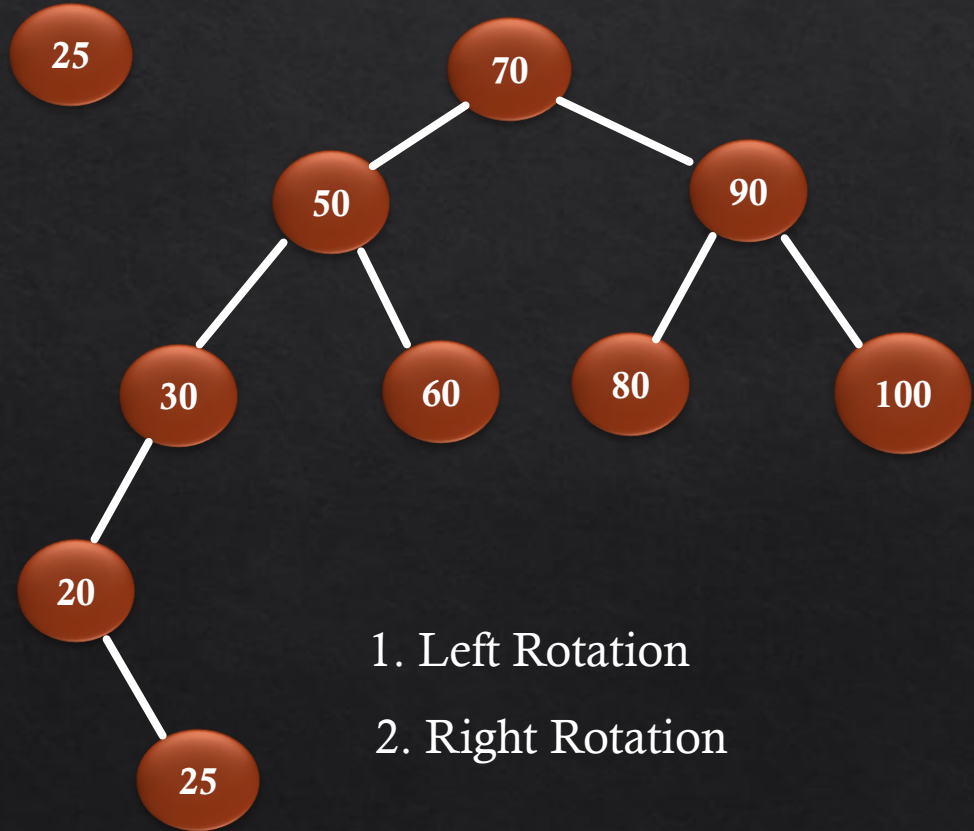
```
def rightRotate(disbalanceNode):  
    newRoot = disbalanceNode.leftChild  
    disbalanceNode.leftChild = disbalanceNode.leftChild.rightChild  
    newRoot.rightChild = disbalanceNode  
    update height of disbalanceNode and newRoot  
    return newRoot
```



Insert a node in an AVL Tree

Case 2: Rotation is required

- Left Right Condition



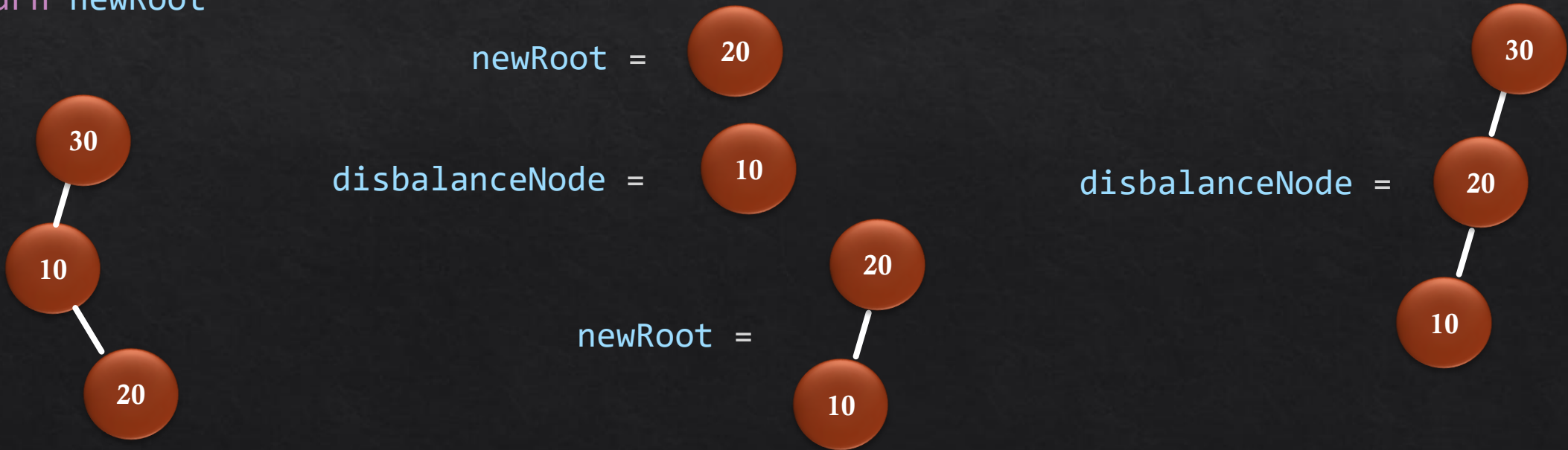
Insert a node in an AVL Tree

Case 2: Rotation is required Left Right Condition

Step 1: rotate Left disbalanceNode.leftChild

Step 2: rotate Right disbalanceNode

```
def leftRotate(disbalanceNode):  
    newRoot = disbalanceNode.rightChild  
    disbalanceNode.rightChild = disbalanceNode.rightChild.leftChild  
    newRoot.leftChild = disbalanceNode  
    update height of disbalanceNode and newRoot  
    return newRoot
```



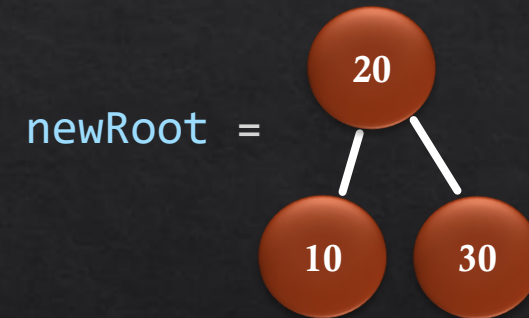
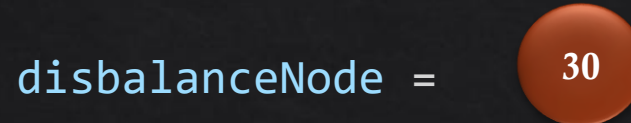
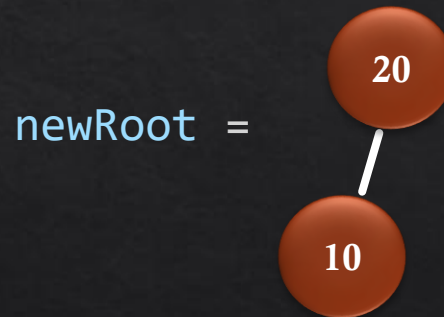
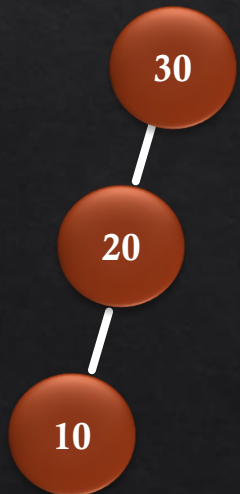
Insert a node in an AVL Tree

Case 2: Rotation is required Left Right Condition

Step 1: rotate left disbalancedNode.leftChild

Step 2: rotate Right disbalancedNode

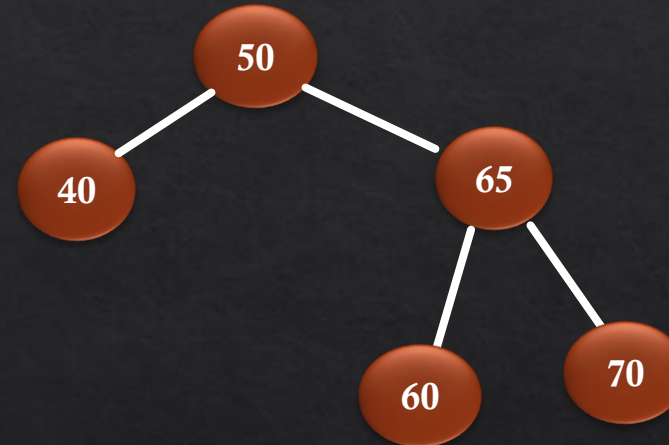
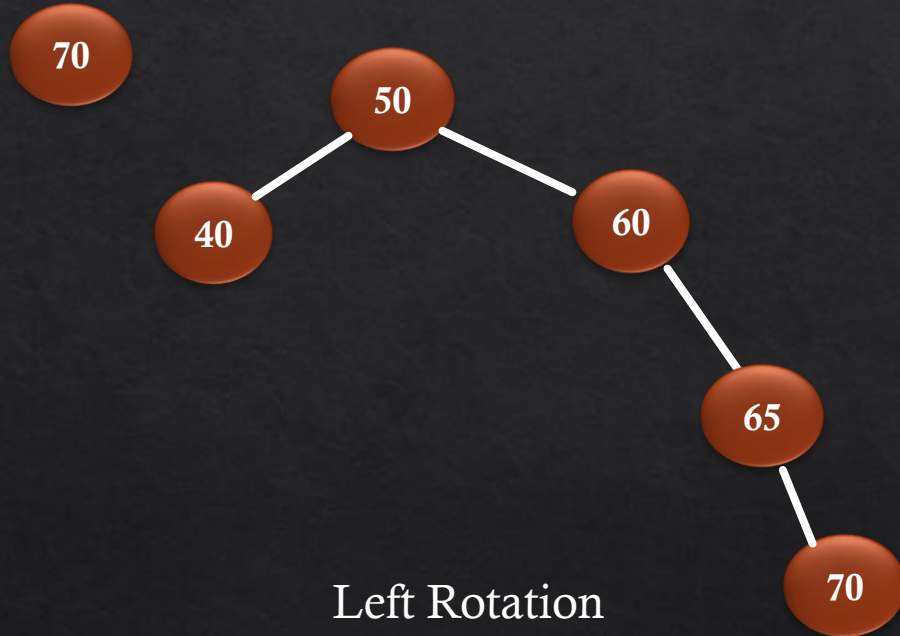
```
def rightRotate(disbalanceNode):  
    newRoot = disbalanceNode.leftChild  
    disbalanceNode.leftChild = disbalanceNode.leftChild.rightChild  
    newRoot.rightChild = disbalanceNode  
    update height of disbalanceNode and newRoot  
    return newRoot
```



Insert a node in an AVL Tree

Case 2: Rotation is required

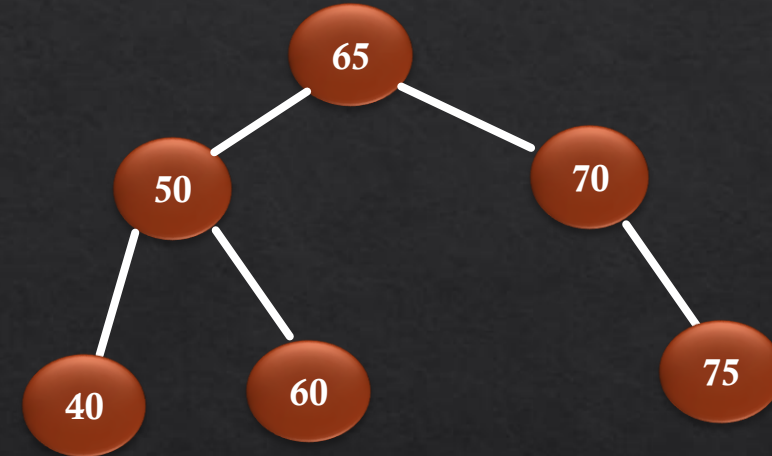
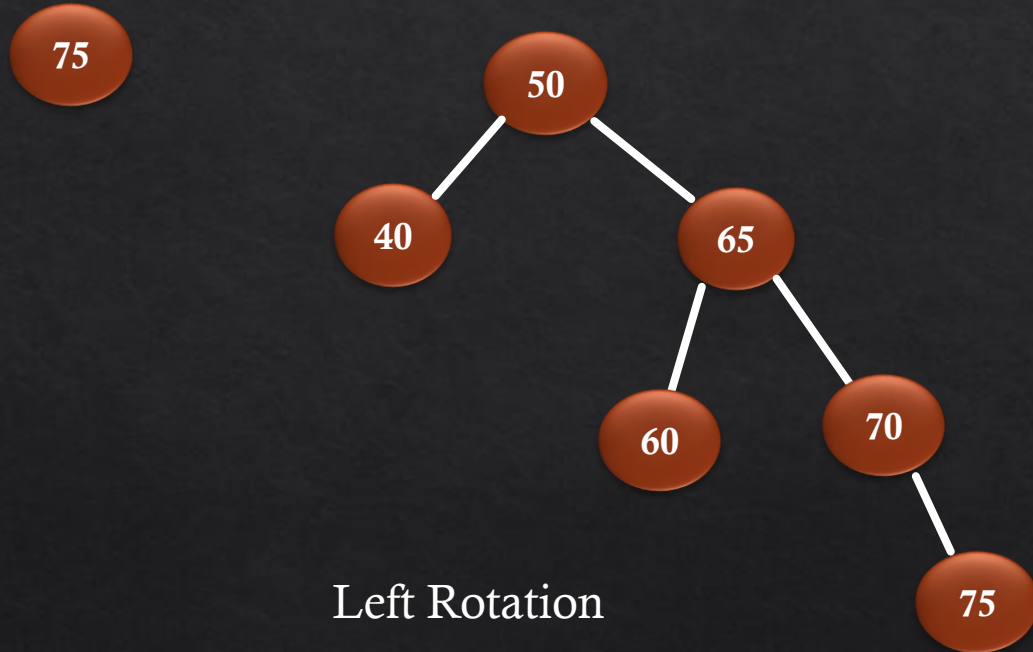
- Right Right Condition Example 1



Insert a node in an AVL Tree

Case 2: Rotation is required

- Right Right Condition Example 2

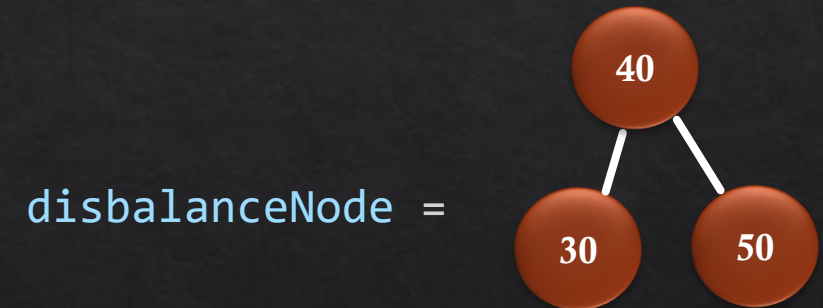
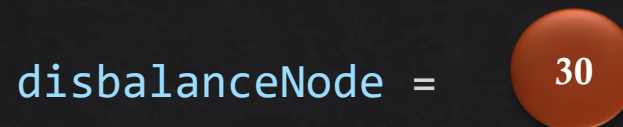
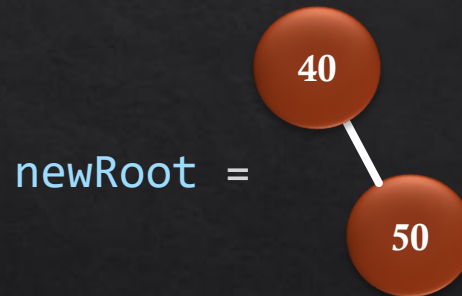
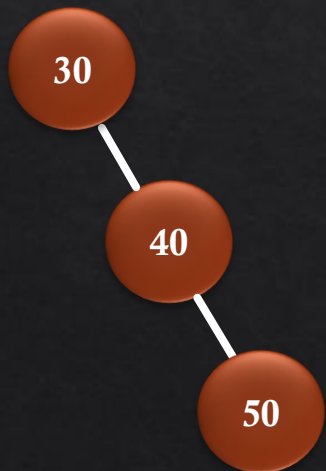


Insert a node in an AVL Tree

Case 2: Rotation is required Right Right Condition

Step 1: rotate Right disbalancedNode

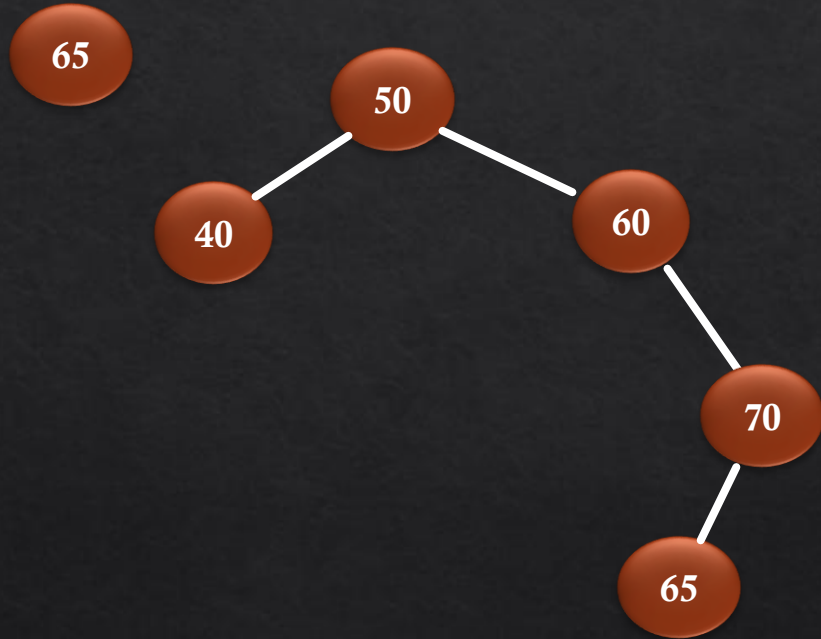
```
def leftRotate(disbalanceNode):  
    newRoot = disbalanceNode.rightChild  
    disbalanceNode.rightChild = disbalanceNode.rightChild.leftChild  
    newRoot.leftChild = disbalanceNode  
    update height of disbalanceNode and newRoot  
    return newRoot
```



Insert a node in an AVL Tree

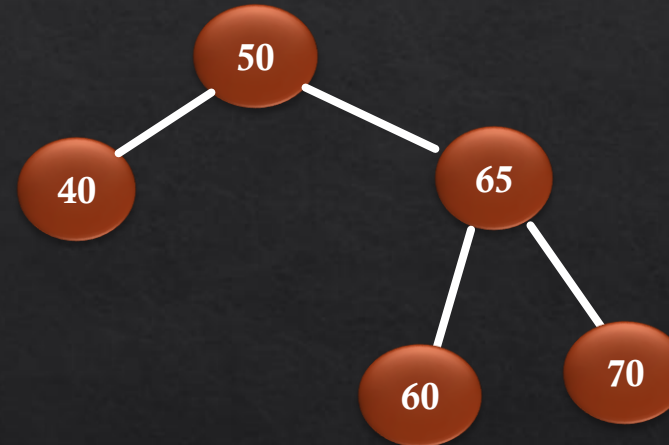
Case 2: Rotation is required

- Right Left Condition



Right Rotation

Left Rotation



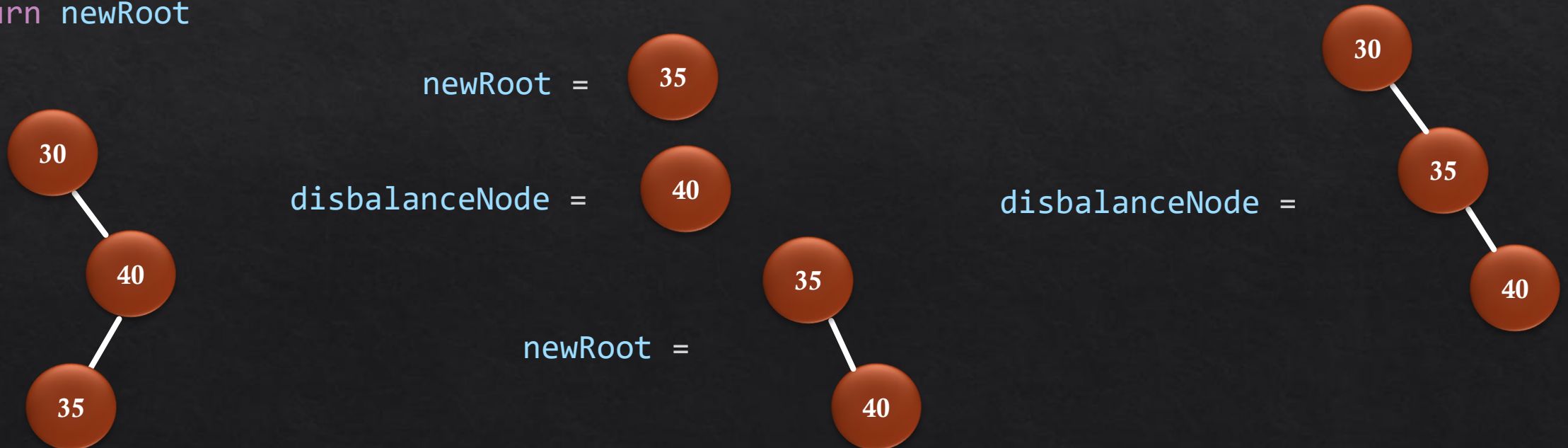
Insert a node in an AVL Tree

Case 2: Rotation is required Right Left Condition

Step 1: rotate Right disbalanceNode.rightChild

Step 2: rotate Left disbalanceNode

```
def rightRotate(disbalanceNode):  
    newRoot = disbalanceNode.leftChild  
    disbalanceNode.leftChild = disbalanceNode.leftChild.rightChild  
    newRoot.rightChild = disbalanceNode  
    update height of disbalanceNode and newRoot  
    return newRoot
```



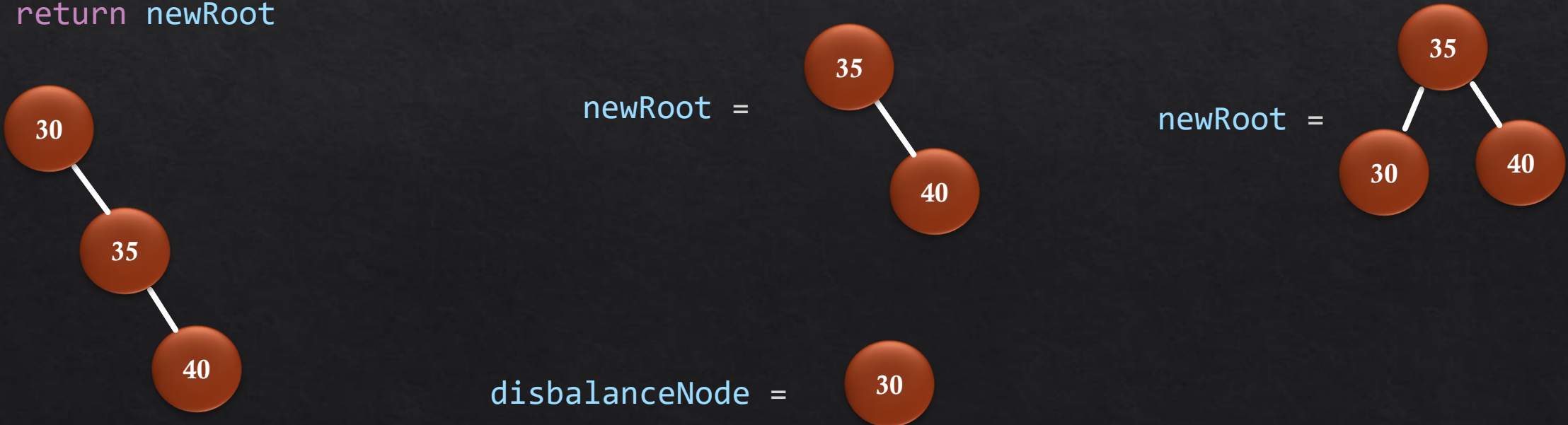
Insert a node in an AVL Tree

Case 2: Rotation is required Left Right Condition

Step 1: rotate left disbalancedNode.leftChild

Step 2: rotate Right disbalancedNode

```
def leftRotate(disbalanceNode):
    newRoot = disbalanceNode.rightChild
    disbalanceNode.rightChild = disbalanceNode.rightChild.leftChild
    newRoot.leftChild = disbalanceNode
    update height of disbalanceNode and newRoot
    return newRoot
```

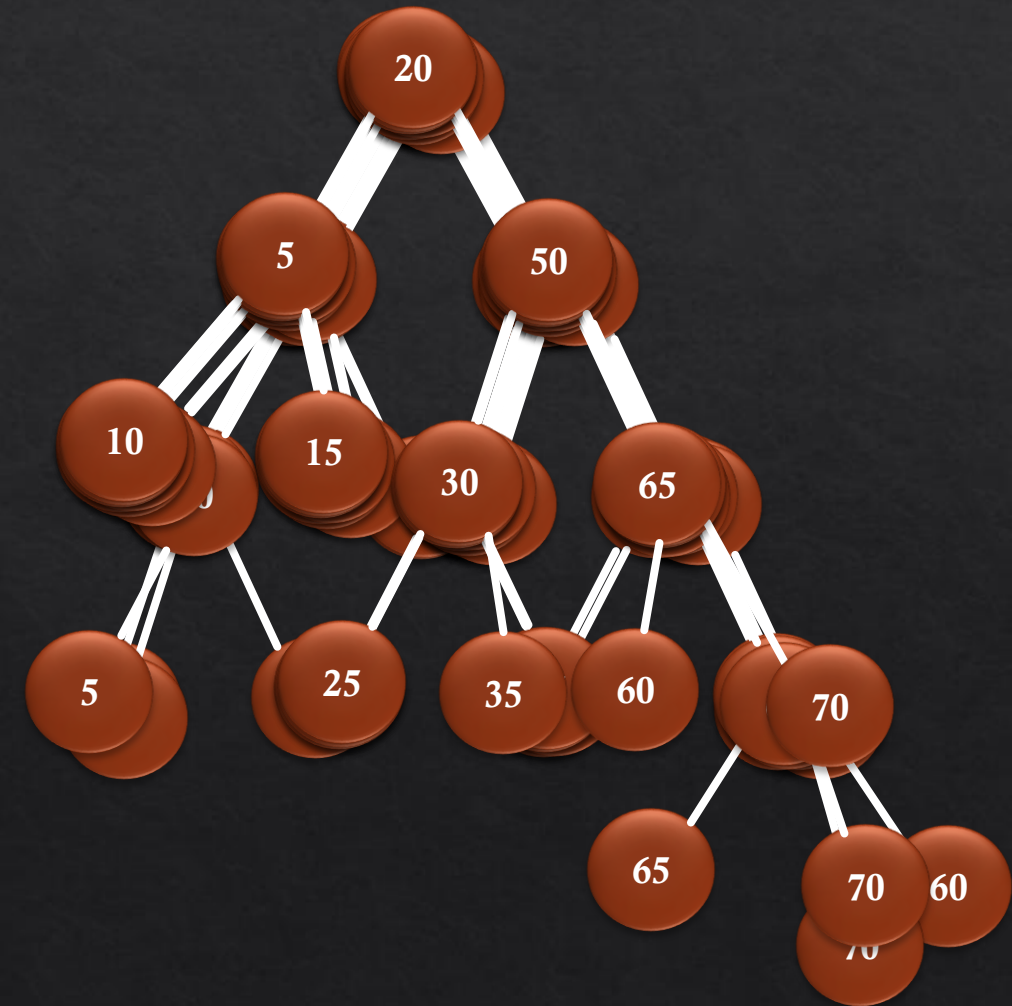


Insert a node in an AVL Tree (All together)

Case 2: Rotation is required

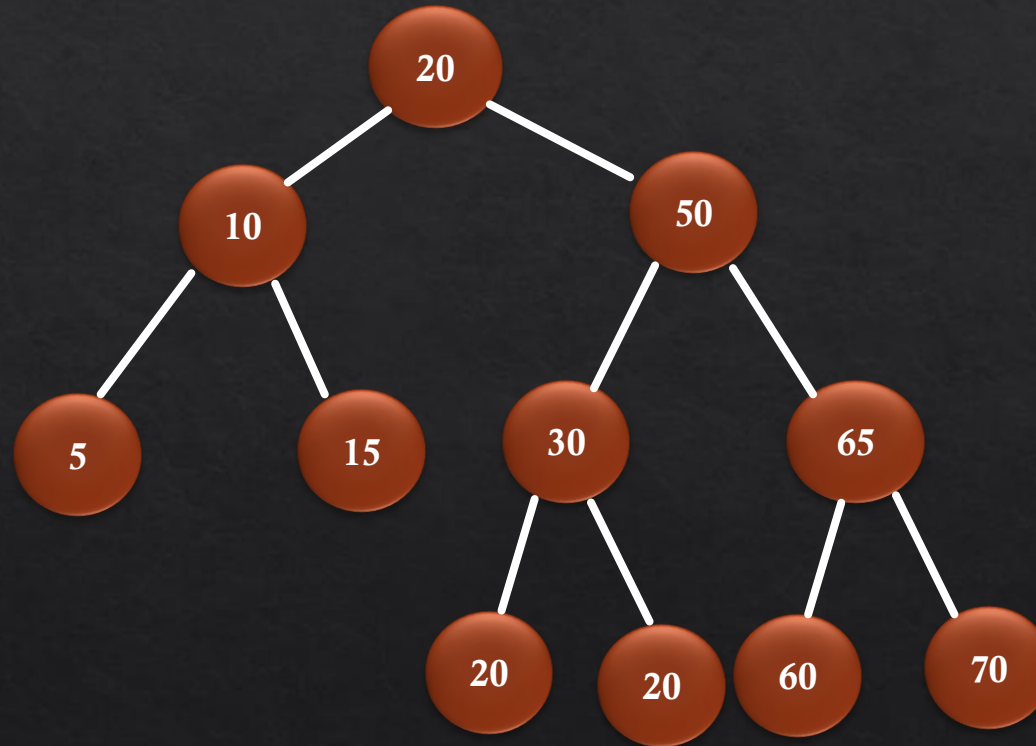
- LL - Left Left Condition
- LR - Left Right Condition
- RR - Right Right Condition
- RL - Right Left Condition

30, 25, 35, 20, 15, 5, 10, 50, 60, 70, 65



Insert a node in an AVL Tree

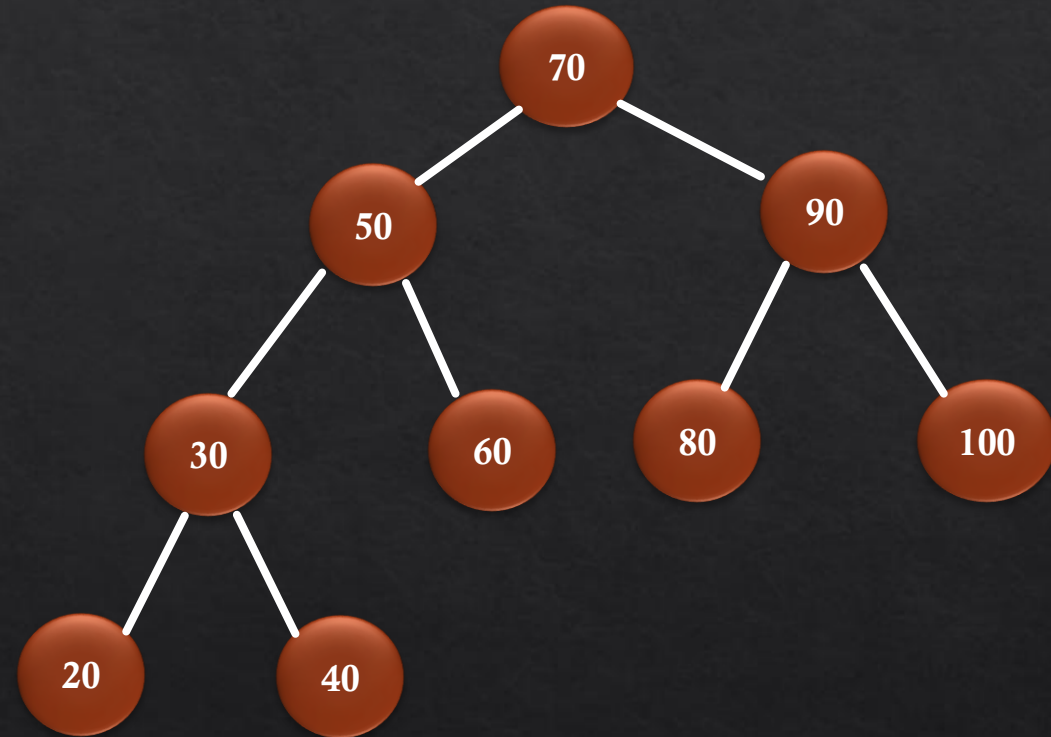
30, 25, 35, 20, 15, 5, 10, 50, 60, 70, 65



Delete a node in an AVL Tree

Case 1: Rotation is not required

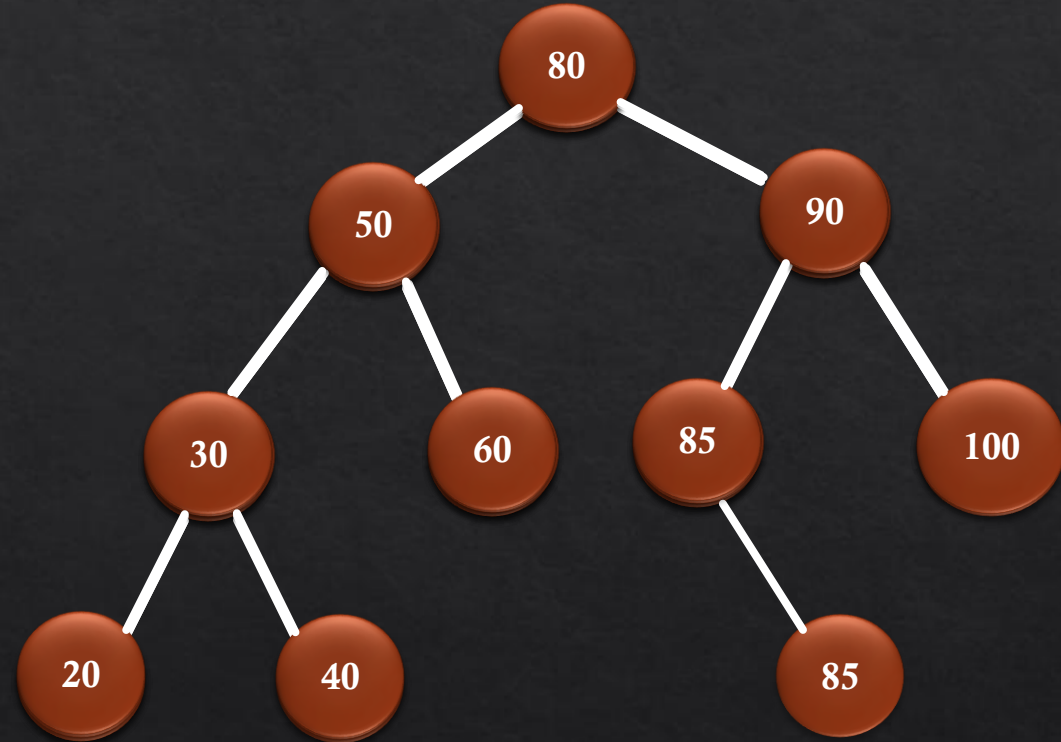
Case 2: Rotation is required (LL, LR, RR, RL)



Delete a node in an AVL Tree

Case 1: Rotation is not required

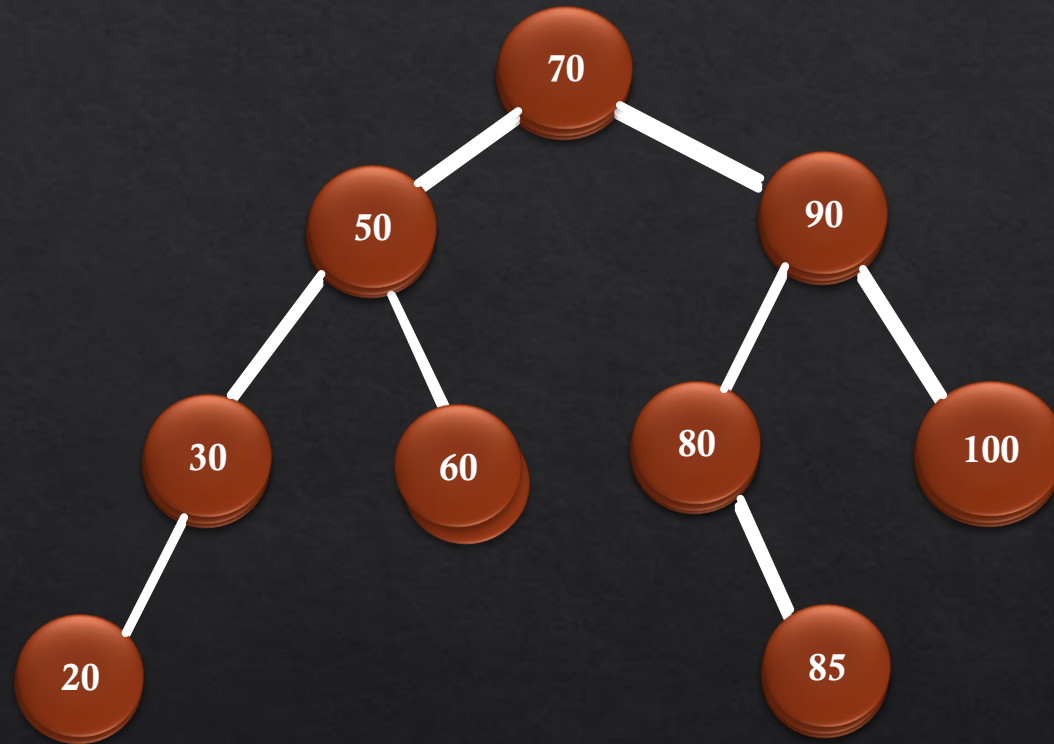
- The node to be deleted is a leaf node
- The node to be deleted has a child node
- The node to be deleted has two children



Delete a node in an AVL Tree

Case 2: Rotation is required

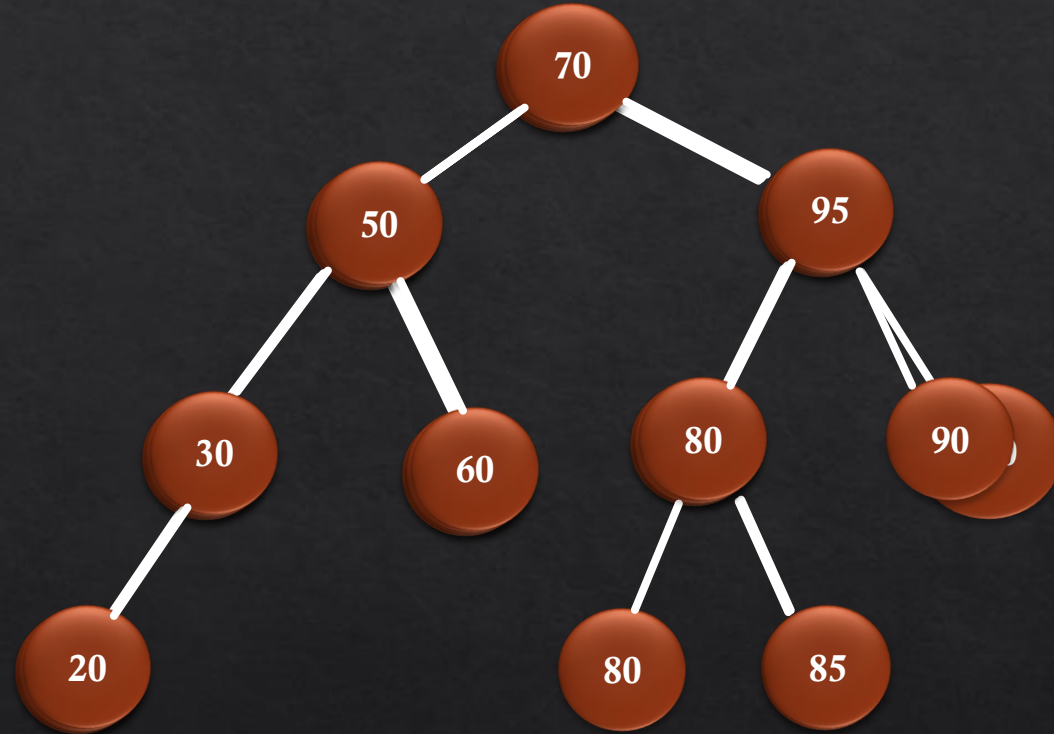
- LL - Left Left Condition



Delete a node in an AVL Tree

Case 2: Rotation is required

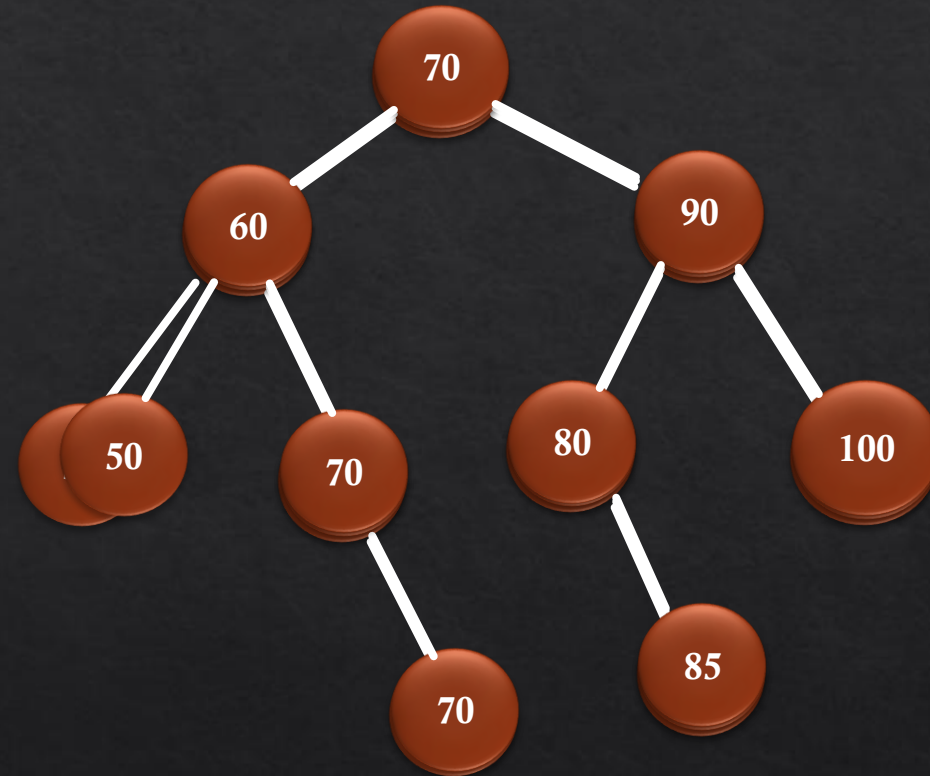
- LR - Left Right Condition



Delete a node in an AVL Tree

Case 2: Rotation is required

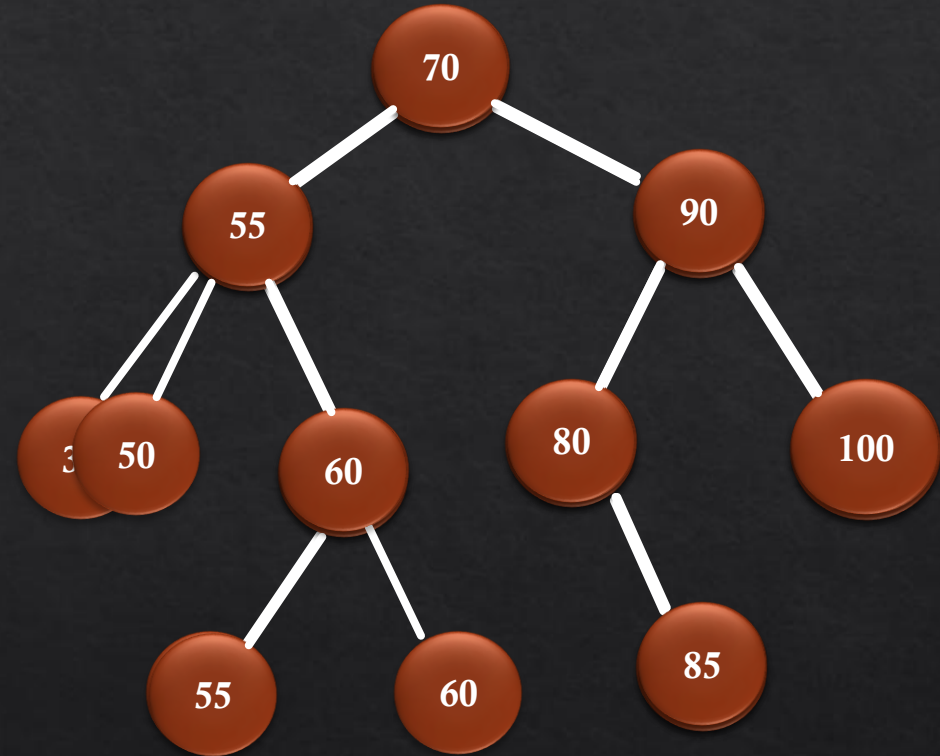
- RR - Right Right Condition



Delete a node in an AVL Tree

Case 2: Rotation is required

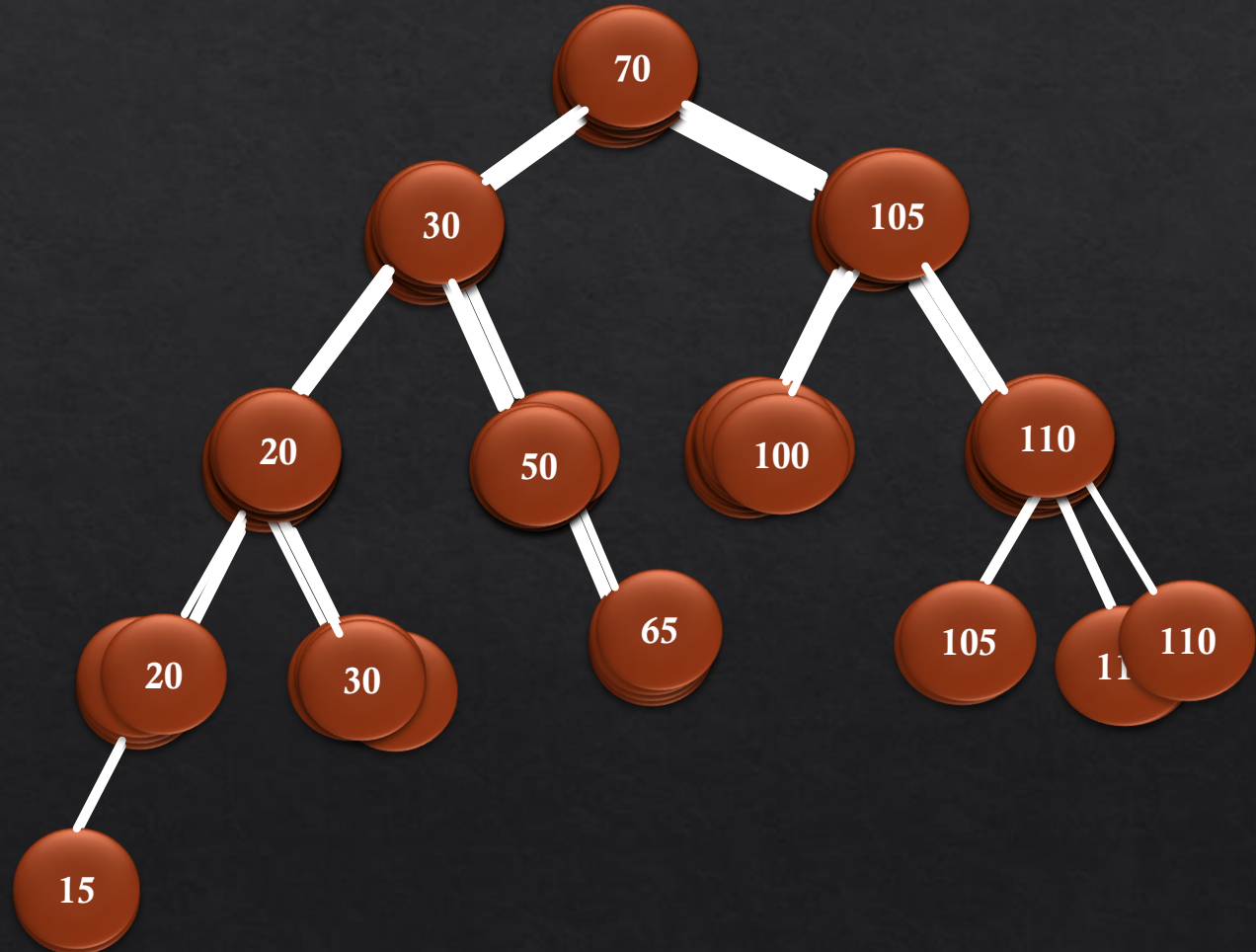
- RL - Right Left Condition



Delete a node in an AVL Tree (All together)

Case 2: Rotation is required

- LL - Left Left Condition
- LR - Left Right Condition
- RR - Right Right Condition
- RL - Right Left Condition



Delete the entire AVL Tree

rootNode = None

rootNode.leftChild = None

rootNode.rightChild = None

